Title

Course Name: Advanced Programming (OOP) Lab

Project Title: **Simulation of Elevator Control System using State Pattern**

Student Name: SNEHA KAR

Roll Number: 75

Instructor Name: Soumadip Biswas

Spring 2026

Problem Statement

Modern buildings rely heavily on elevator systems to efficiently transport people between floors. Managing elevator operations involves handling multiple behavioural states such as idle, moving up, moving down, and door operations while responding to user requests from different floors. Traditional implementations often rely on complex conditional statements, making the system difficult to maintain, understand, and extend.

The proposed Elevator Control System Simulation addresses this challenge by modeling elevator behaviour using the State Design Pattern. Each operational state of the elevator will be represented by a separate class encapsulating its specific behaviour. This allows the elevator to dynamically change its behaviour when its internal state changes.

To improve clarity and realism, the system will include more granular door states, such as DoorOpenState, DoorClosingState, and DoorClosedState, ensuring a more accurate representation of elevator operations and transitions. Additionally, the system will provide a clearly defined real-time status display (console-based or simple Java Swing GUI) to visualize elevator movement and state changes.

This approach demonstrates strong object-oriented principles such as abstraction, modularity, and separation of concerns.

System Design

The **Elevator Control System** is designed using the **State Design Pattern**, where the behaviour of the elevator changes dynamically based on its current state.

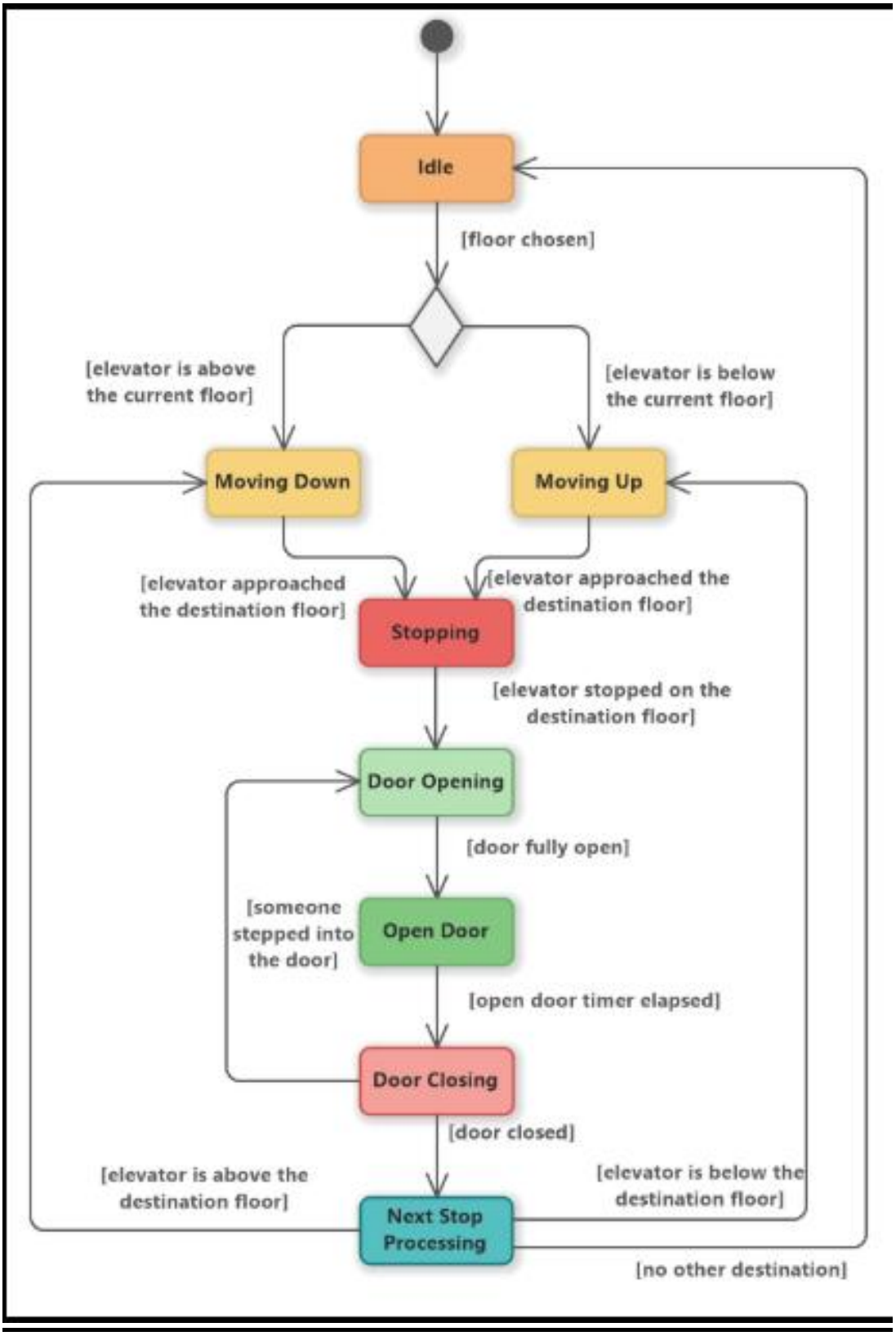
The system follows a **modular and layered design**:

- The **Elevator class (Context)** maintains the current state and floor.
- A **State interface** defines common operations like handling requests and movement.
- Multiple **Concrete State classes** implement specific behaviors:
  - IdleState
  - MovingUpState
  - MovingDownState
  - DoorOpenState
  - DoorClosingState
  - DoorClosedState
- A **Request Queue** manages floor requests efficiently.
- A **UI Layer (Console or Swing GUI)** displays real-time status.

This design ensures:

- Separation of concerns
- Easy scalability (add more states easily)
- Reduced complexity (no large if-else blocks)

UML Class Diagram



Description of Major Classes

1. Elevator (Context Class)

- Maintains:
  - Current floor
  - Current state
  - Request queue
- Delegates behaviour to current state
- Changes state dynamically

---

2. State (Interface)

- Defines common methods:
  - handleRequest()
  - move()
- Ensures all states follow a common structure

---

3. Concrete State Classes

IdleState

- Waits for requests
- Decides next movement

MovingUpState

- Moves elevator upwards
- Stops at requested floors

MovingDownState

- **Moves elevator downwards**

DoorOpenState

- **Opens door for passengers**

DoorClosingState

- **Handles door closing transition**

DoorClosedState

- **Prepares elevator for next movement**

---

4. RequestQueue

- **Stores floor requests**
- **Can use Queue / PriorityQueue**
- **Helps in scheduling**

---

5. UI Layer (Console / Swing GUI)

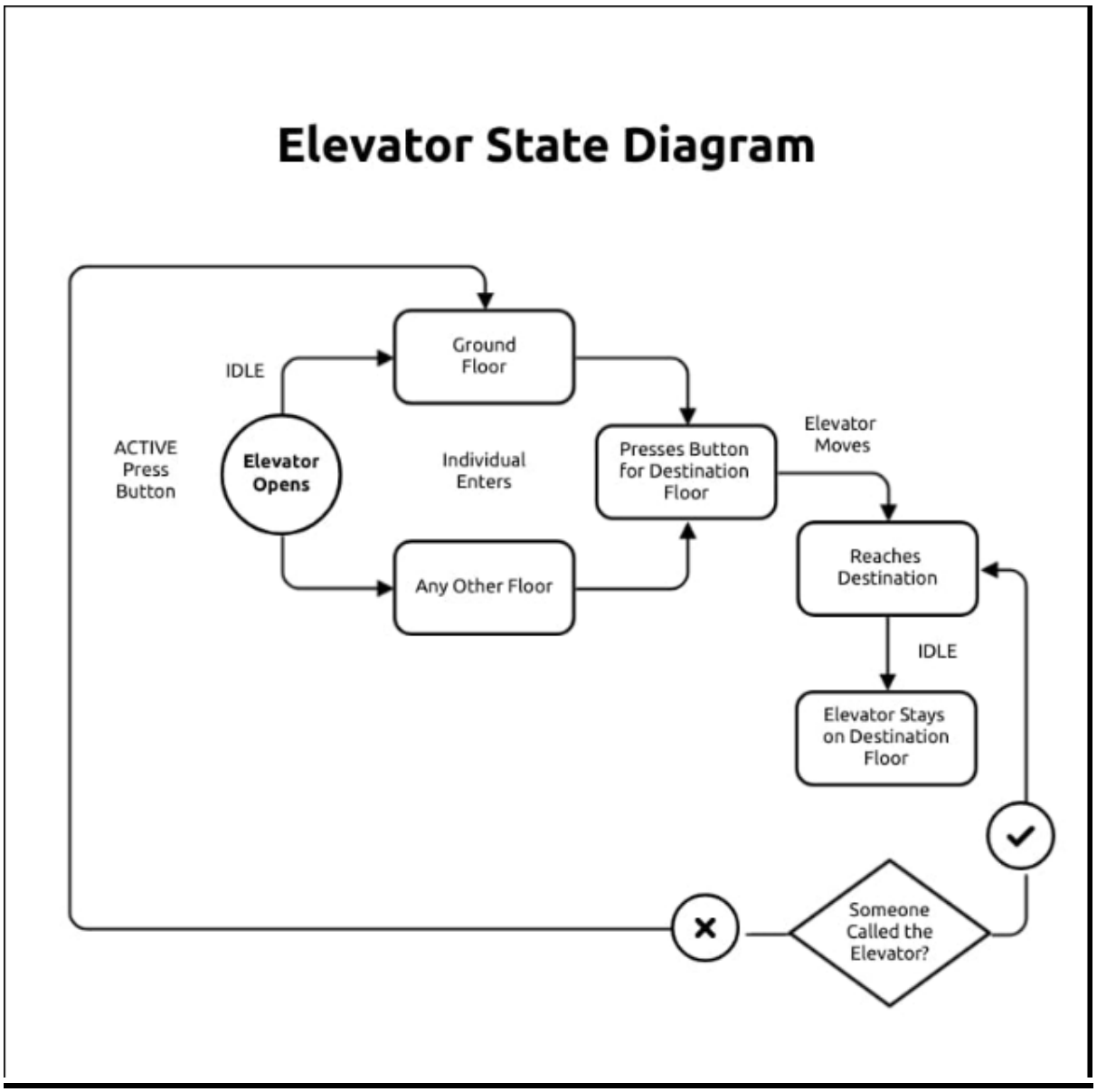
- **Displays:**
  - **Current floor**
  - **Current state**
- **Takes user input**

Package Structure

A clean package structure improves readability and modularity.

```
com.elevator

│── main
│    └── Main.java

│── model
│    └── Elevator.java

│── state
│    ├── State.java
│    ├── IdleState.java
│    ├── MovingUpState.java
│    ├── MovingDownState.java
│    ├── DoorOpenState.java
│    ├── DoorClosingState.java
│    └── DoorClosedState.java

│── service
│    └── RequestQueue.java

│── ui
│    └── ElevatorUI.java
```

Interaction Flow



Step-by-Step Flow

1. User requests elevator (selects floor)
2. Request is added to RequestQueue
3. Elevator checks current state

Flow Example

IdleState

↓

Receive Request

↓

MovingUpState / MovingDownState

↓

Reach Floor

↓

DoorOpenState

↓

DoorClosingState

↓

DoorClosedState

↓

Back to IdleState (or next request)

OOP Concepts Demonstrated

1. Abstraction

Where it is used

- In the State interface

Why it is used

- To define common behaviour for all elevator states without exposing implementation details
- Ensures every state follows the same structure

Code Snippet

```
public interface State {
    void handleRequest(Elevator elevator, int floor);
    void move(Elevator elevator);
}
```

Explanation

- State defines *what* operations should be performed
- It does not define *how* — that is done by concrete states
- This hides complexity and enforces a contract for all states

2. <u>Inheritance</u>

Where it is used

- Concrete classes like IdleState, MovingUpState, etc.

Why it is used

- To allow all states to **reuse the common interface**
- Enables code extensibility (new states can be added easily)

Code Snippet

```
public class IdleState implements State {
    @Override
    public void handleRequest(Elevator elevator, int floor) {
        if (floor > elevator.getCurrentFloor()) {
            elevator.setState(new MovingUpState());
        }
    }

    @Override
    public void move(Elevator elevator) {
        System.out.println("Elevator is idle");
    }
}
```

Explanation

- IdleState **implements** State
- It provides its own version of behaviour
- This shows inheritance of structure and contract

3. Polymorphism

Where it is used

- When Elevator calls methods on State without knowing the exact class

Why it is used

- To allow **different behaviour for same method call**
- Removes need for if-else conditions

Code Snippet

```
public class Elevator {
   private State currentState;

   public void setState(State state) {
      this.currentState = state;
   }

   public void requestFloor(int floor) {
      currentState.handleRequest(this, floor);
   }
}
```

Explanation

- currentState can be **any state object**
- Same method call → different behaviour
- Example:
    - IdleState → starts moving
    - MovingState → continues movement

4. Encapsulation

Where it is used

- In the Elevator class (data + methods together)

Why it is used

- To protect internal data
- Control how state and floor are modified

Code Snippet

```
public class Elevator {
    private int currentFloor;
    private State currentState;

    public int getCurrentFloor() {
        return currentFloor;
    }

    public void setState(State state) {
        this.currentState = state;
    }
}
```

Explanation

- Variables are private
- Access is controlled through methods
- Prevents invalid or direct modification

5. <u>Exception Handling</u>

Where it is used

- While handling invalid floor requests

Why it is used

- To make system **robust and error-safe**
- Prevent crashes due to wrong input

Code Snippet

```
public void requestFloor(int floor) {
    if (floor < 0 || floor > 10) {
        throw new IllegalArgumentException("Invalid floor!");
    }
    currentState.handleRequest(this, floor);
}
```

Explanation

- Checks for invalid input
- Throws exception if input is incorrect
- Ensures system reliability

6. Collections

Where it is used

- For storing multiple floor requests

Why it is used

- To manage requests efficiently
- Supports scheduling and ordering

Code Snippet

```
import java.util.*;

public class RequestQueue {
    private Queue<Integer> queue = new LinkedList<>();

    public void addRequest(int floor) {
        queue.offer(floor);
    }

    public int getNextRequest() {
        return queue.poll();
    }
}
```

Explanation

- Uses **Queue** to store requests
- offer() adds request
- poll() retrieves next request
- Helps simulate real elevator scheduling

Implementation Highlights

1. State Transition Logic (Core of State Pattern)

Code Snippet

```
public class Elevator {
    private State currentState;

    public void setState(State state) {
        this.currentState = state;
    }

    public void requestFloor(int floor) {
        currentState.handleRequest(this, floor);
    }
}
```

Justification

This is the heart of the State Pattern

The elevator does not use if-else conditions

Behavior changes dynamically based on currentState

Makes the system clean, flexible, and scalable

2. Concrete State Behavior (Moving Up)

Code Snippet

```
public class MovingUpState implements State {

    @Override
    public void handleRequest(Elevator elevator, int floor) {
        System.out.println("Already moving up...");
    }

    @Override
    public void move(Elevator elevator) {
        elevator.incrementFloor();
        System.out.println("Moving up to floor: " +
elevator.getCurrentFloor());

        if (elevator.isTargetReached()) {
            elevator.setState(new DoorOpenState());
        }
    }
}
```

Justification

Encapsulates specific behavior of moving upward

Removes logic from main class → improves modularity

State decides when to transition (to DoorOpenState)

Makes code easy to extend and debug

3. Request Queue Handling

Code Snippet

```
import java.util.*;

public class RequestQueue {
    private Queue<Integer> queue = new LinkedList<>();

    public void addRequest(int floor) {
        queue.offer(floor);
    }

    public int getNextRequest() {
        return queue.poll();
    }
}
```

Justification

- Efficiently manages multiple floor requests
- Uses built-in Queue (FIFO) → realistic behaviour
- Separates scheduling logic from Elevator → clean design
- Easily replaceable with PriorityQueue for advanced version

4. Door State Transition (Improved Design)

Code Snippet

```
public class DoorOpenState implements State {

    @Override
    public void handleRequest(Elevator elevator, int floor) {
        System.out.println("Door is open, please wait...");
    }

    @Override
    public void move(Elevator elevator) {
        System.out.println("Closing door...");
        elevator.setState(new DoorClosingState());
    }
}
```

Justification

- Implements granular door control
- Separates door logic from movement logic
- Makes transitions realistic and easy to understand
- Improves clarity in state flow and UML diagram

Testing & Error Handling

1. Test Cases Considered

These test cases validate normal working of the elevator system:

Basic Functional Tests

- Request elevator from current floor
- Request elevator from a higher floor (Up movement)
- Request elevator from a lower floor (Down movement)
- Multiple floor requests (queue handling)
- Door operations after reaching a floor

State Transition Tests

- Idle → MovingUp
- Idle → MovingDown
- Moving → DoorOpen
- DoorOpen → DoorClosing → DoorClosed → Next state

Sample Test Case

Test Case	Input	Expected Output
TC1	Request floor 5 from floor 1	Elevator moves up
TC2	Request floor 2 from floor 5	Elevator moves down
TC3	Reach target floor	Door opens

2. <u>Edge Cases Handled</u>

These ensure the system behaves correctly in unusual but possible situations:

Boundary Conditions

- Minimum floor (e.g., 0 or Ground floor)
- Maximum floor limit (e.g., 10 or N floors)

Repeated Requests

- Same floor requested multiple times
- Ignored or handled efficiently

Empty Queue

- No pending requests → Elevator stays in **IdleState**

Rapid Multiple Inputs

- Multiple floor requests in quick succession
- Stored and processed in queue

Door State Conflicts

- Prevent movement when door is open
- Ensure proper sequence: Open → Close → Move

3.Failure Scenarios

These cover invalid or unexpected situations:

Invalid Input Handling

```
public void requestFloor(int floor) {
    if (floor < 0 || floor > maxFloor) {
        throw new IllegalArgumentException("Invalid floor number!");
    }
    currentState.handleRequest(this, floor);
}
```

Prevents system crash due to wrong input

Null State Protection

```
if (currentState == null) {
    throw new IllegalStateException("Elevator state not initialized!");
}
```

Ensures system is properly initialized before use

Empty Queue Handling

```
if (requestQueue.isEmpty()) {
    System.out.println("No pending requests. Elevator is idle.");
    setState(new IdleState());
}
```

Prevents unnecessary movement

Unexpected State Behavior

- If state transition fails → system resets to **IdleState**
- Prevents system from getting stuck

Git Workflow

Commits on Mar 29, 2026

Update README.md Sneha-Kar2005 authored 1 hour ago	Verified	51af3b2
Update README.md Sneha-Kar2005 authored 52 minutes ago	Verified	80b38dc
Update README.md Sneha-Kar2005 authored 51 minutes ago	Verified	e32e0db
Create State Interface.java Sneha-Kar2005 authored 37 minutes ago	Verified	b541280
Delete src/.keep Sneha-Kar2005 authored 36 minutes ago	Verified	fdffec8
Create Elevator Class.java Sneha-Kar2005 authored 34 minutes ago	Verified	761d12a
Create Idle State.java Sneha-Kar2005 authored 33 minutes ago	Verified	cc7cf14
Create Moving Up State.java Sneha-Kar2005 authored 32 minutes ago	Verified	a354d4c
Create Door Open State.java Sneha-Kar2005 authored 31 minutes ago	Verified	2be7d0e
Create Moving Down State.java Sneha-Kar2005 authored 24 minutes ago	Verified	4ca5d49
Create Door Closing State.java Sneha-Kar2005 authored 22 minutes ago	Verified	d58b98b
Create Door Closed State.java Sneha-Kar2005 authored 22 minutes ago	Verified	0c5b021
Create Request Queue.java Sneha-Kar2005 authored 21 minutes ago	Verified	f60cac7
Create Main Class.java Sneha-Kar2005 authored 20 minutes ago	Verified	4d9d930

Explanation of Development Progression

Initial Setup:

The repository was created with basic folders such as src, docs, report, and slides. A .keep file was initially used to maintain folder structure.

Project Structure Design:

The source folder was organized into packages (main, model, state, service) to follow proper object-oriented design principles.

Core Implementation:

The State Design Pattern was implemented by creating the State interface and concrete state classes such as IdleState, MovingUpState, MovingDownState, and door-related states.

- **Feature Development:**
  Elevator functionalities such as floor request handling, movement simulation, and door operations were added. A queue-based system was implemented to manage multiple requests.

- **Enhancements:**
  Additional improvements like DoorClosingState and DoorClosedState were introduced for better state modeling. Error handling for invalid inputs was also implemented.

- **Documentation:**
  The README file, report, and supporting documents were added and refined to clearly explain the system design and implementation.

- **Finalization:**
  The project was reviewed, tested, and finalized before submission.

Conclusion

The **Simulation of Elevator Control System using State Pattern** successfully demonstrates the application of object-oriented design principles in modeling a real-world system. By using the **State Design Pattern**, the elevator's behaviour dynamically changes based on its current state, eliminating complex conditional logic and improving code maintainability.

The system effectively handles floor requests, simulates elevator movement, and models door operations through well-defined states such as Idle, Moving, and Door-related states. The modular architecture ensures separation of concerns, making the system easy to understand, extend, and debug.

Overall, this project provides a practical understanding of design patterns, system design, and clean coding practices.

Future Scope

The current system can be further enhanced with the following improvements:

- Multi-Elevator System:
  Extend the design to manage multiple elevators with optimized scheduling.
- Priority-Based Scheduling:
  Implement intelligent algorithms to prioritize requests based on distance, direction, or urgency.
- Graphical User Interface (GUI):
  Develop a full-featured GUI with animations for better visualization.
- Real-Time Simulation:
  Add timing delays to simulate real-world elevator movement.
- Smart Elevator Optimization:
  Use AI/ML-based techniques to improve efficiency and reduce waiting time.
- Emergency Handling:
  Include features like fire mode, overload detection, and emergency stops.
- Persistence Layer:
  Store elevator logs and request history using file or database storage.